

Chat Context Export

Platform: ChatGPT

Exported: 13/09/2026, 15:27:11

Conversation ID: 6a990fb7-bef8-83ee-a5eb-c40230f2b7a8

Conversation

User

****Project Description****: Build a tool that passively captures TLS ClientHello and ServerHello messages and computes JA3/JA3S fingerprints (with JA4/JA4S as a stretch goal) to identify the client or server application/library generating the traffic. Curate a small reference database and demonstrate distinguishing real clients (browsers, curl, custom scripts) purely from their handshake fingerprint.

****Tools/Technologies****: : libpcap/Scapy (or raw sockets) for capture; fingerprint computation implemented per the JA3/JA4 specifications, eBPF/XDP, Packet/Flow Generators like TRex, KV-stores (Redis, Valkey).

****Expected Outcome****: Demonstrate the working JA3/JA3S fingerprint extractor, validated against published reference JA3 hashes for known clients; A curated database correctly identifying at least 5 distinct clients/tools by fingerprint alone. Demonstrate distinguishing, e.g., curl vs. a browser vs. a custom TLS client on the wire. Understanding on the fingerprinting's role in security monitoring (malware C2 detection, client identification) and its limitations, including fingerprint randomization in modern browsers as an evasion technique. My JA3, JA3S, database to some extent and the code of capturing and converting in cpp is written, I need to test it vigorously. I already have make, I need you to tell me all the kind of edge tests that we need to implement

Assistant

Yes. Since your JA3 + JA3S computation, C++ packet capture/conversion, and database are already written, the next step should be a proper adversarial test suite, not just "does curl give the expected hash?" For this project, I would test at 5 levels: Packet/parser edge cases JA3/JA3S correctness edge cases Capture/network edge cases Real-client integration tests Database + identification + performance tests The important thing is that you want to prove both correctness and robustness.

1. First: Build a golden test suite Before the weird edge cases, establish a set of packets where the expected result is known. For every test, ideally store: For example: This is much better than testing only: because if your hash is wrong, you need to know which field caused it.
2. TLS packet structure tests This should be your biggest category.
 - 2.1 Minimum valid ClientHello Test the smallest syntactically valid ClientHello you can construct. Verify: parser doesn't crash correct TLS version correct cipher suites correct extension list correct curves correct point formats correct JA3 string correct MD5
 - 2.2 ClientHello with no extensions Very important. A ClientHello can have: Your output should correctly handle: or whatever the specification requires for empty fields. Test:
 - 2.3 No elliptic curves extension Have extensions but omit: Your parser shouldn't accidentally reuse values from a previous packet.
 - 2.4 No EC point formats Same idea. Test: and:

3. GREASE tests — VERY IMPORTANT This is probably one of the most important edge cases for JA3. Modern TLS clients frequently contain GREASE values. Examples include values such as: Your JA3 implementation must ignore GREASE values where the JA3 specification requires them to be ignored. Test all of these: Test A GREASE cipher suite. Test B GREASE extension. Test C GREASE supported group. Test D GREASE appearing multiple times. Test E GREASE at beginning. Test F GREASE at end. Test G Multiple different GREASE values. Test H Same GREASE value repeated. Most importantly: Construct two ClientHellos that differ only in GREASE values and verify that they produce the same JA3. This is an excellent project demonstration.

4. Ordering tests JA3 is sensitive to ordering. Create: and: They should produce different JA3 fingerprints. Do the same for: cipher suites extensions supported groups EC point formats Then test that your parser preserves wire order. This catches a common bug: or sorting vectors accidentally before generating JA3.

5. Duplicate values Construct packets containing: and: Check that you don't silently deduplicate values unless the specification requires it. JA3 is derived from the ordered lists appearing in the handshake.

6. TLS version tests Test different ClientHello versions: TLS 1.0 TLS 1.1 TLS 1.2 TLS 1.3 TLS 1.3 deserves special attention because the ClientHello legacy version and the supported_versions extension interact differently. Test: and verify your JA3 implementation uses the correct field according to the JA3 specification.

7. TLS 1.3 tests You absolutely need several TLS 1.3 captures. Test: TLS 1.3 ClientHello TLS 1.3 ServerHello TLS 1.3 with GREASE TLS 1.3 with many extensions TLS 1.3 with minimal extensions TLS 1.3 with key_share TLS 1.3 with supported_versions TLS 1.3 with PSK-related extensions Also test clients that negotiate: versus: and verify that your extractor handles both.

8. Extension ordering This is another excellent test. Create: and: The fingerprints should differ if the resulting JA3 extension sequence differs. Also test: Your parser should still extract the fingerprint rather than failing.

9. Unknown extensions Create fake extensions: with: zero-length body small body large body Verify: and the unknown extension is handled according to the JA3 extraction rules. This is important because real TLS implementations evolve.

10. Malformed packet tests This is where you test whether your C++ program is actually robust. You should deliberately corrupt packets. Truncate ClientHello Cut it at: Every truncation should result in: and not a segmentation fault. Corrupt record length For example: Your parser must not read beyond the packet buffer. Corrupt handshake length Same idea: Corrupt extension length For example: Corrupt individual extension length For example: Invalid cipher-suite length Cipher suite list length should be even. Test: Your parser should reject it safely. Invalid extension length Test odd/unexpected lengths where the structure requires specific sizes.

11. Empty packet tests Feed: Your program should simply say: rather than crash.

12. Random garbage / fuzz tests This one is high priority for C++. Generate random byte sequences: and feed them into the parser. Requirement: No crashes, no hangs, no out-of-bounds access. Even better, compile with: and run your fuzz tests. If you use CMake eventually, you can have a sanitizer build. Also run: These can find bugs that normal tests completely miss.

13. Ethernet/IP/TCP capture tests Your extractor isn't receiving clean TLS bytes. It receives packets. So test different network encapsulations. Ethernet Normal: IPv6 This is essential.

14. VLAN packets Test: Your Ethernet/IP offset cannot simply be hardcoded.

15. Double VLAN / QinQ If you want a really strong parser: Check that it doesn't assume:

16. IPv4 options Most basic implementations assume: Test IPv4 packets with options. Your TCP header offset should be calculated

using the actual IP header length. 17. TCP options Test packets containing different TCP options: timestamps window scaling SACK MSS These shouldn't affect TLS extraction. 18. TCP segmentation Very important. A TLS ClientHello does not necessarily arrive in one TCP packet. For example: Your capture code must reconstruct enough TCP stream data to recognize the ClientHello. Test: This is one of the tests I'd consider essential for your project. 19. TLS record fragmentation Even more specifically, test: where the handshake is split across TLS records. Your extractor shouldn't assume: 20. TCP retransmission Capture a retransmitted packet. Example: Your stream reconstruction should not accidentally parse: as one malformed handshake. 21. TCP out-of-order packets Create: This is an excellent robustness test. A passive network monitor needs to deal with this. If your current implementation assumes packets always arrive in sequence, document that limitation or implement reassembly. 22. Multiple simultaneous connections Capture: Your program needs to keep flows separate. Test using: as the flow identity. 23. Same client, multiple connections For example: The JA3 should remain consistent where the underlying TLS configuration is the same. 24. ClientHello retransmission / duplicate handshake Test whether you accidentally count the same ClientHello twice. 25. ServerHello tests Do the same for JA3S. Test: TLS 1.2 ServerHello TLS 1.3 ServerHello different cipher suites different extensions no extensions GREASE unknown extensions malformed ServerHello truncated ServerHello fragmented ServerHello And especially: Two servers using different configurations should generate different JA3S. 26. Real client matrix This is where your project becomes convincing. I'd make a table like:

Client	TLS library	Expected JA3	Detected Chrome	BoringSSL	Firefox	NSS	Safari	Apple					
TLX	curl	OpenSSL	Python	requests	OpenSSL	custom	C++	OpenSSL	custom	Python	Python	SSL	TLX

You need at least the five required by your project, but I'd aim for 8–10. 27. curl variations Don't test only: Test: Different curl builds may use different TLS libraries. That's actually an excellent demonstration: Application name alone doesn't uniquely determine JA3; the TLS library/configuration matters. 28. Browser variations Test at least: Chrome Firefox Safari If possible, test different versions. Then investigate whether the fingerprints remain stable. This directly supports your project's discussion about modern browser fingerprint randomization/evasion. 29. Python clients Test: then: then potentially: You may discover that multiple applications have the same underlying TLS fingerprint. That's actually a useful finding. 30. Custom TLS client Create a tiny custom program: and another: Then show: or: depending on configuration. This is important for discussing the limitations of fingerprinting. 31. Database tests Your database should also be tested independently. Exact match Unknown fingerprint Collision / multiple labels If two applications share a fingerprint: Don't assume: 32. Database normalization tests Test: Your database interface should behave predictably. 33. Persistence tests If you're using Redis/Valkey/KV storage, test: insert lookup update duplicate insert missing key restart database corrupted/missing value connection failure timeout database unavailable Your packet capture program shouldn't crash just because Redis is down. 34. Cache tests If you cache fingerprints: verify that behavior. Also test cache invalidation if applicable. 35. False-positive tests This is extremely important for the project's actual security claim. Take: and ask: Can another TLS client intentionally reproduce it? Try configuring a custom OpenSSL client to approximate the same ClientHello. You don't necessarily need to perfectly clone it, but demonstrating that JA3 identifies TLS behavior rather than cryptographically proving application identity

is valuable. 36. False-negative tests Same client but modified configuration: If fingerprints differ: That demonstrates the limitation from the opposite direction. 37. Fingerprint stability Run the same client: Record: Calculate: For deterministic clients: For randomized clients: This would make a very nice experimental result for your report. 38. Cross-machine tests Run the same client on: Compare fingerprints. This tells you whether your database is actually identifying the application/library or merely one particular machine/build. 39. Capture interface tests Test: If available. Also test: depending on how your capture is implemented. 40. Non-TLS traffic Feed it: Expected: No false extraction. 41. TLS-looking garbage Create traffic beginning with: but which isn't actually TLS. This is important. Your detector shouldn't simply say: It should validate enough of the TLS structure. 42. Wrong TLS handshake type Test: Your JA3 extractor should only process the appropriate handshake. For example: 43. TLS alerts Feed: including: and verify the program handles it cleanly. 44. Very large ClientHello Generate a ClientHello with a very large extension set. Test: This catches: fixed-size buffer assumptions integer overflow stack buffer problems incorrect length handling 45. Integer overflow tests Particularly important in C++. Try lengths near: and malformed values such as: while the actual packet is tiny. You want: rather than: 46. Endianness tests Make sure multi-byte TLS fields are interpreted correctly. Test known values such as: and verify they aren't accidentally interpreted little-endian. 47. PCAP-specific tests Have captures with: empty PCAP single packet many packets truncated PCAP corrupted PCAP different link-layer types large PCAP multiple simultaneous flows Your tool should fail gracefully on corrupted PCAPs. 48. Performance tests Once correctness is established, test: Measure: Also measure: because a passive monitor shouldn't continuously accumulate packets. 49. Memory leak tests Run your extractor against a huge capture under: and/or: Look for: 50. Long-running capture test Run: or ideally: while generating TLS traffic continuously. Check: Memory should remain approximately stable. This catches flow-table leaks. 51. Flow-table stress test Generate: simultaneous/different flows if feasible. Check: memory lookup speed cleanup timeout behavior 52. Timeout tests If you maintain TCP flows, test: Then: and make sure it doesn't corrupt state. 53. Concurrency tests If your implementation uses multiple threads: test under heavy traffic. Run with: if applicable. You want to catch: race conditions double frees shared-state corruption 54. Database failure during packet capture This is an important real-world test. Start capture: Then kill Redis. Expected: Your entire packet-processing pipeline shouldn't die. 55. Database recovery Then: Verify the system automatically resumes. 56. Reference-hash validation This should be a dedicated test category. For every known fingerprint: Maintain a table: TestExpectedActualStatusChromehash Ahash APASSFirefoxhash Bhash BPASScurlhash Chash CPASSPythonhash Dhash DPASS This is one of the strongest things you can show your professor. 57. Differential testing This is one of the best tests you can implement. Take the same PCAP and run it through: and an independent implementation/library. Then: for hundreds/thousands of captures. Don't rely solely on your own implementation's expected outputs because that can create: "testing the implementation against itself." 58. Property-based tests You can generate valid/invalid TLS structures automatically. Useful properties: Property 1 Adding GREASE shouldn't change JA3. Property 2 Reordering cipher suites should generally change JA3. Property 3 Changing a non-JA3-relevant field shouldn't change JA3. Property 4 Changing a JA3-relevant field should change JA3. Property 5 Malformed lengths never crash the parser. Property 6 Random bytes never crash

the parser. These are excellent automated tests. 59. Regression tests Every bug you encounter should become: For example: Then: should run everything. 60. The test suite I'd actually build Don't make 60 separate C++ test programs. Structure it something like: And ultimately: should produce something like: ¶ If you want to prioritize Don't try to implement everything at once. I'd rank them: ¶ Must-have Published JA3 reference captures GREASE TLS 1.2 TLS 1.3 IPv4 IPv6 TCP fragmentation Malformed lengths Truncated packets Random/fuzz input curl Chrome Firefox Python/custom client JA3S reference tests Unknown/non-TLS traffic ¶ Very valuable TCP retransmission Out-of-order TCP VLAN IPv4 options Multiple simultaneous flows Database failure/recovery Memory leak testing Long-running capture Performance testing Differential testing against another JA3 implementation ¶ Excellent for making the project stand out Property-based testing Browser fingerprint stability experiment Same JA3 ¶ multiple applications Same application ¶ multiple JA3s Attempted fingerprint impersonation JA3 randomization experiment Large-scale PCAP replay Sanitizer + fuzzing campaign One particularly important point Because you're doing passive packet capture in C++, I'd separate your testing into: Test each layer independently. Otherwise, when one Chrome PCAP gives the wrong hash, you won't know whether the problem is Ethernet parsing, TCP reassembly, TLS parsing, JA3 construction, MD5, or the database. If you upload your current C++ source + Makefile (and especially your existing test files, if any), I can go through the actual implementation and turn this into a concrete test checklist mapped to your functions, including the exact edge cases your current code is vulnerable to and a structure.

Code Block 1 (text)

```
input.pcap
expected_clienthello_fields.json
expected_ja3_string
expected_ja3_hash
expected_serverhello_fields.json
expected_ja3s_string
expected_ja3s_hash
```

Code Block 2 (text)

```
{
  "tls_version": 771,
  "cipher_suites": [4865, 4866, 4867],
  "extensions": [0, 11, 10, 35, 16, 5, 13],
  "elliptic_curves": [29, 23, 24],
  "ec_point_formats": [0],
  "ja3": "...",
  "ja3_hash": "..."
}
```

Code Block 3 (text)

```
pcap -> hash -> compare hash
```

Code Block 4 (text)

```
cipher_suites
compression_methods
extensions = empty
```

Code Block 5 (text)

```
JA3 = version,ciphers,,,
```

Code Block 6 (text)

```
extensions_length = 0
```

Code Block 7 (text)

```
supported_groups
```

Code Block 8 (text)

```
supported_groups present
ec_point_formats absent
```

Code Block 9 (text)

```
ec_point_formats present
supported_groups absent
```

Code Block 10 (text)

```
0x0a0a
0x1a1a
0x2a2a
...
```

Code Block 11 (text)

```
GREASE, TLS_AES_128_GCM_SHA256, ...
```

Code Block 12 (text)

```
ClientHello A
cipher suites = A,B,C
```

Code Block 13 (text)

```
ClientHello B
cipher suites = C,B,A
```

Code Block 14 (text)

```
std::set
```

Code Block 15 (text)

```
cipher = A,A,B
```

Code Block 16 (text)

```
extensions = X,X,Y
```

Code Block 17 (text)

```
0x0301
```

Code Block 18 (text)

```
0x0302
```

Code Block 19 (text)

```
0x0303
```

Code Block 20 (text)

```
legacy_version = TLS 1.2  
supported_versions = TLS 1.3
```

Code Block 21 (text)

```
TLS 1.2
```

Code Block 22 (text)

```
TLS 1.3
```

Code Block 23 (text)

```
extensions:  
SNI  
supported_groups  
signature_algorithms
```

Code Block 24 (text)

```
extensions:  
signature_algorithms  
SNI
```

supported_groups

Code Block 25 (text)

unknown extension

Code Block 26 (text)

type = 0x1234

Code Block 27 (text)

parser succeeds

Code Block 28 (text)

1 byte
5 bytes
10 bytes
20 bytes
...

Code Block 29 (text)

invalid/incomplete

Code Block 30 (text)

actual length = 100
declared length = 1000

Code Block 31 (text)

Handshake length > available bytes

Code Block 32 (text)

extensions_length = 5000
actual bytes = 20

Code Block 33 (text)

extension_length = 100
actual extension data = 5

Code Block 34 (text)

length = odd number

Code Block 35 (text)

empty file

Code Block 36 (text)

empty packet

Code Block 37 (text)

zero-byte payload

Code Block 38 (text)

Ethernet-only packet

Code Block 39 (text)

IP-only packet

Code Block 40 (text)

TCP packet with no payload

Code Block 41 (text)

not TLS ClientHello

Code Block 42 (text)

1 byte
2 bytes
...
10 KB
100 KB
1 MB

Code Block 43 (text)

-fsanitize=address,undefined

Code Block 44 (text)

AddressSanitizer
UndefinedBehaviorSanitizer

Code Block 45 (text)

Ethernet ▯ IPv4 ▯ TCP ▯ TLS

Code Block 46 (text)

Ethernet ▯ IPv6 ▯ TCP ▯ TLS

Code Block 47 (text)

Ethernet
▯
802.1Q VLAN
▯
IPv4
▯
TCP
▯
TLS

Code Block 48 (text)

Ethernet
▯ VLAN
▯ VLAN
▯ IPv4
▯ TCP
▯ TLS

Code Block 49 (text)

IP starts at byte 14

Code Block 50 (text)

IPv4 header = 20 bytes

Code Block 51 (text)

TCP packet 1:
[ClientHello first 50 bytes]

TCP packet 2:
[remaining ClientHello]

Code Block 52 (text)

ClientHello in 1 TCP packet
ClientHello in 2 packets
ClientHello in 3 packets

ClientHello in 10 packets

Code Block 53 (text)

TLS record 1
TLS record 2

Code Block 54 (text)

one TLS record = one ClientHello

Code Block 55 (text)

packet 1
packet 2
packet 2 retransmission
packet 3

Code Block 56 (text)

ClientHello + ClientHello

Code Block 57 (text)

packet 1
packet 3
packet 2

Code Block 58 (text)

Client A ↔ Server A
Client B ↔ Server B
Client A ↔ Server C
Client C ↔ Server B

Code Block 59 (text)

src IP
src port
dst IP
dst port
protocol

Code Block 60 (text)

curl ↔ server 1
curl ↔ server 2
curl ↔ server 3

Code Block 61 (text)

```
curl https://example.com
```

Code Block 62 (text)

```
curl --tlsv1.2 ...  
curl --tlsv1.3 ...
```

Code Block 63 (text)

```
ssl
```

Code Block 64 (text)

```
requests
```

Code Block 65 (text)

```
urllib3
```

Code Block 66 (text)

```
OpenSSL
```

Code Block 67 (text)

```
different TLS configuration
```

Code Block 68 (text)

```
same application  
different JA3
```

Code Block 69 (text)

```
different application  
same JA3
```

Code Block 70 (text)

```
JA3 = known fingerprint  
🔍 Chrome
```

Code Block 71 (text)

```
JA3 = abc123...  
🔍 UNKNOWN
```

Code Block 72 (text)

```
JA3 {application A, application B}
```

Code Block 73 (text)

```
one fingerprint = exactly one application
```

Code Block 74 (text)

```
uppercase hash  
lowercase hash  
leading/trailing whitespace  
newline  
empty string  
invalid MD5
```

Code Block 75 (text)

```
first packet { database lookup  
second packet { cache
```

Code Block 76 (text)

```
Chrome JA3
```

Code Block 77 (text)

```
curl configuration A  
curl configuration B
```

Code Block 78 (text)

```
same application  
different JA3
```

Code Block 79 (text)

```
100 times
```

Code Block 80 (text)

```
JA3_1  
JA3_2  
...  
JA3_100
```

Code Block 81 (text)

number of unique JA3s

Code Block 82 (text)

1 unique fingerprint

Code Block 83 (text)

multiple fingerprints

Code Block 84 (text)

Machine A
Machine B
Machine C

Code Block 85 (text)

loopback
Wi-Fi
Ethernet

Code Block 86 (text)

promiscuous mode
non-promiscuous mode

Code Block 87 (text)

HTTP
DNS
SSH
FTP
ICMP
random TCP
UDP

Code Block 88 (text)

not TLS

Code Block 89 (text)

16 03 03 ...

Code Block 90 (text)

first byte == 0x16

ClientHello

Code Block 91 (text)

ClientHello
ServerHello
Certificate
Finished
Alert

Code Block 92 (text)

Certificate don't calculate JA3

Code Block 93 (text)

TLS Alert

Code Block 94 (text)

fatal
warning

Code Block 95 (text)

1 KB
4 KB
8 KB
16 KB

Code Block 96 (text)

UINT16_MAX

Code Block 97 (text)

65535

Code Block 98 (text)

safe rejection

Code Block 99 (text)

buffer overflow

Code Block 100 (text)

0x0303
0x1301
0x1302
0x1303

Code Block 101 (text)

1 MB PCAP
10 MB
100 MB
1 GB

Code Block 102 (text)

packets/sec
MB/sec
CPU usage
memory usage

Code Block 103 (text)

peak memory

Code Block 104 (text)

AddressSanitizer

Code Block 105 (text)

LeakSanitizer

Code Block 106 (text)

definitely lost
indirectly lost

Code Block 107 (text)

30 minutes

Code Block 108 (text)

several hours

Code Block 109 (text)

RAM at t=0
RAM at t=30m

RAM at t=1h

Code Block 110 (text)

10,000
50,000
100,000

Code Block 111 (text)

flow created
☐ no packets
☐ timeout
☐ flow deleted

Code Block 112 (text)

old flow packet arrives

Code Block 113 (text)

capture thread
parser threads
database thread

Code Block 114 (text)

ThreadSanitizer

Code Block 115 (text)

capture ☐ JA3 ☐ Redis

Code Block 116 (text)

capture continues
fingerprint extraction continues
database lookup reports failure

Code Block 117 (text)

Redis down
☐
Redis comes back
☐
new fingerprints

Code Block 118 (text)

known PCAP

⌘

your extractor

⌘

JA3

⌘

expected published JA3

Code Block 119 (text)

Your JA3 implementation

Code Block 120 (text)

your JA3 == reference JA3

Code Block 121 (text)

tests/regression/

Code Block 122 (text)

regression_001_grease.pcap

regression_002_ipv6.pcap

regression_003_fragmented_clienthello.pcap

regression_004_bad_extension_length.pcap

Code Block 123 (text)

make test

Code Block 124 (text)

tests/

⌘ ⌘ unit/

⌘ ⌘ ⌘ test_ja3.cpp

⌘ ⌘ ⌘ test_ja3s.cpp

⌘ ⌘ ⌘ test_grease.cpp

⌘ ⌘ ⌘ test_extensions.cpp

⌘ ⌘ ⌘ test_parser.cpp

⌘

⌘ ⌘ malformed/

⌘ ⌘ ⌘ truncated_clienthello.pcap

⌘ ⌘ ⌘ bad_length.pcap

⌘ ⌘ ⌘ bad_extension.pcap

⌘ ⌘ ⌘ garbage.pcap

⌘

⌘ ⌘ fragmentation/

⌘ ⌘ ⌘ one_packet.pcap

⌘ ⌘ ⌘ two_packets.pcap

```
fragmented_tls.pcap
out_of_order.pcap
clients/
chrome.pcap
firefox.pcap
safari.pcap
curl.pcap
python.pcap
custom_client.pcap
server/
serverhello_tls12.pcap
serverhello_tls13.pcap
regression/
...
```

Code Block 125 (text)

```
make test
```

Code Block 126 (text)

```
=====
JA3/JA3S Test Suite
=====

[PASS] Basic ClientHello
[PASS] Empty Extensions
[PASS] GREASE Cipher Suite
[PASS] GREASE Extension
[PASS] GREASE Supported Group
[PASS] Cipher Ordering
[PASS] Extension Ordering
[PASS] TLS 1.2
[PASS] TLS 1.3
[PASS] IPv4
[PASS] IPv6
[PASS] VLAN
[PASS] TCP Fragmentation
[PASS] TCP Retransmission
[PASS] Malformed Length
[PASS] Truncated Packet
[PASS] Random Input
[PASS] Chrome Reference
[PASS] Firefox Reference
[PASS] curl Reference
[PASS] Python Reference
[PASS] JA3S Reference

-----
```

Tests: 47
Passed: 47
Failed: 0

Code Block 127 (text)

Raw PCAP / Data
Packet Extraction
TCP Reassembly
TLS Parsing
JA3 / JA3S
Database Lookup